

Microsoft Fabric Git Integration + CI/CD Pipeline + Multi-Environment Deployment

Lab Overview

In this hands-on implementation lab, you will build a complete enterprise-style DevOps workflow for Microsoft Fabric.

You will:

- Connect a Fabric workspace with Git
- Enable version control for Fabric artifacts
- Build CI/CD pipeline for deployment
- Create Dev and Prod environments
- Deploy Fabric artifacts across environments
- Track changes using Git commits
- Validate deployments in Production

This guide follows a real-world enterprise implementation approach.

Business Scenario

A retail analytics company is building enterprise data solutions on Microsoft Fabric.

The organization wants:

- Development team to work in Dev workspace
- Production workspace for business users
- Full Git-based version control
- CI/CD automation for deployments
- Controlled release management
- Rollback capability
- Collaboration between developers

You are the Fabric DevOps Engineer responsible for implementing the complete solution.

Learning Objectives

After completing this lab, you will be able to:

- Configure Git integration in Microsoft Fabric
- Connect Fabric workspace with Git Repository Git repository
- Commit Fabric artifacts to Git
- Create deployment pipeline in Fabric
- Promote changes from Dev to Prod
- Validate deployments
- Understand enterprise Fabric DevOps workflow

Architecture Flow

Developer Changes



Fabric Dev Workspace



Git Integration Enabled



Git Repository Repository



Deployment Pipeline



Production Workspace

Prerequisites

Required Access

You must have:

- Microsoft Fabric enabled account
- Power BI Premium / Fabric Capacity access

-
- Git Repository access
 - Workspace Admin permission in Fabric
-

Tools Used

Tool	Purpose
Microsoft Fabric	Data platform
Git Repository	Git repository + CI/CD
Git Integration	Version control
Deployment Pipeline	Environment promotion

Lab Environment Setup

Step 1 — Create Dev Workspace

Objective

Create a development workspace where developers will build Fabric artifacts.

Step-by-Step

Open Microsoft Fabric

Open:

Microsoft Fabric : <https://app.fabric.microsoft.com>

Create Workspace

Click:

Workspaces → New Workspace

Configure Workspace

Setting	Value
Workspace Name	Fabric-Dev-Workspace
License Mode	Fabric Capacity
Description	Development Workspace

Click Create

Workspace creation will complete.

Expected Output

Fabric-Dev-Workspace created successfully

Step 2 — Create Production Workspace**Objective**

Create production environment.

Create Another Workspace

Use same process.

Configure Workspace

Setting	Value
Workspace Name	Fabric-Prod-Workspace
License Mode	Fabric Capacity
Description	Production Workspace

Expected Output

Fabric-Prod-Workspace created successfully

Step 3 — Create Git Repository

Objective

Create Git repository for storing Fabric artifacts.

Supported Git Platforms

Microsoft Fabric supports Git integration with:

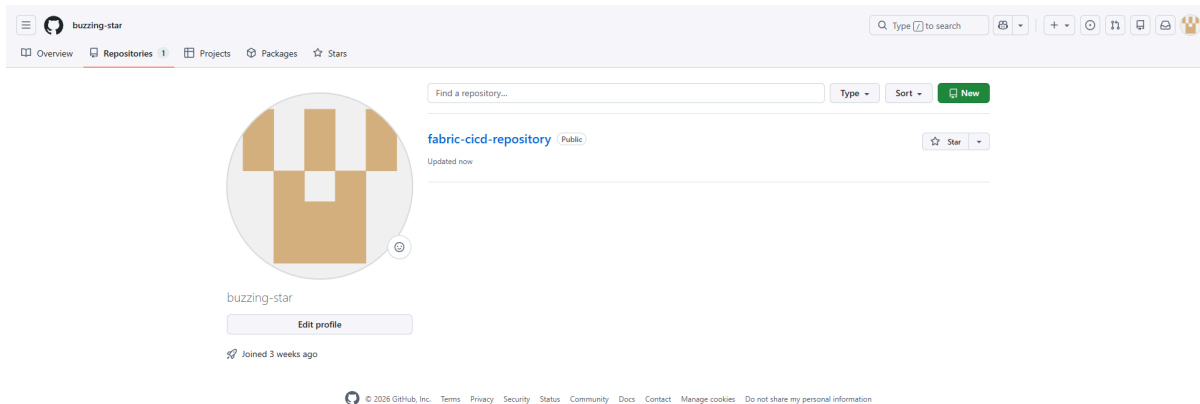
- GitHub
- Git-based repositories

In this lab, we will use Git repository integration only.

Create Repository

Create a repository named:

`fabric-cicd-repository`



Repository Structure

fabric-cicd-repository
└── fabric-artifacts

Expected Output

Git repository created successfully

Step 4 — Connect Fabric Workspace with Git

Objective

Enable Git integration.

Open Fabric Dev Workspace

Go to:

Fabric-Dev-Workspace

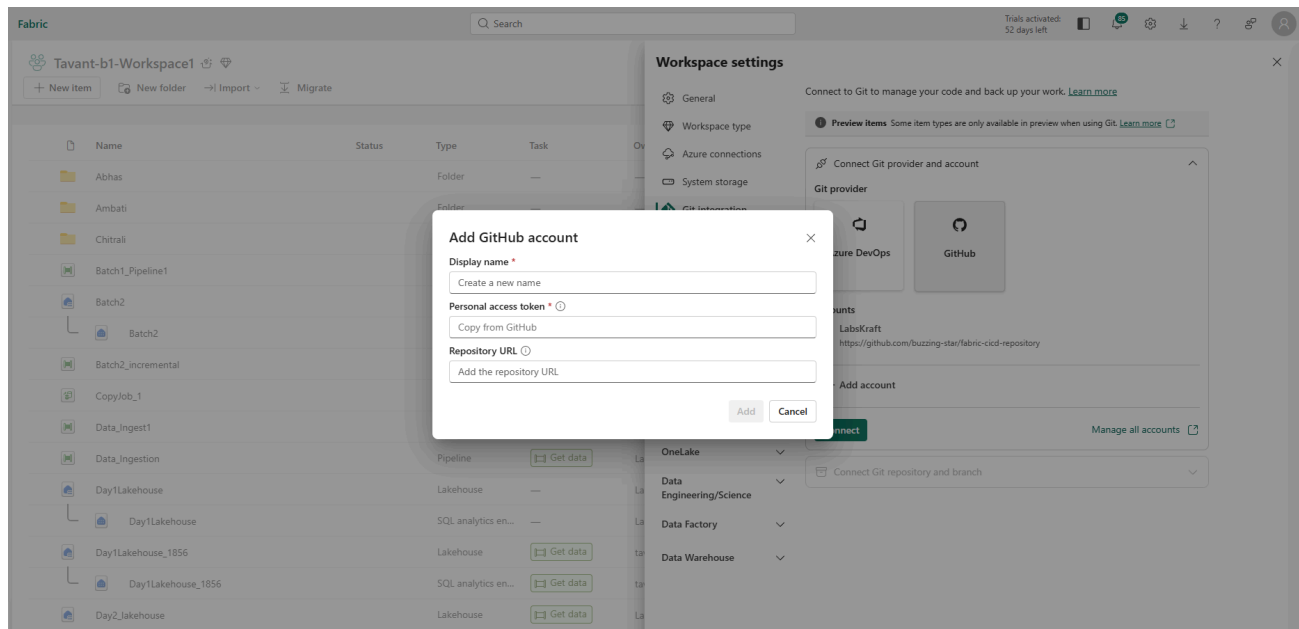
Open Workspace Settings

Click:

Workspace Settings → Git Integration

Configure Git Integration

Setting	Value
Git Provider	GitHub
Repository	fabric-cicd-repository
Display Name	StackRoute
PAT	Get it from GITHUB



Authenticate

Sign in with Git account.

Click Connect and Sync

Expected Output

Workspace connected to Git successfully

Objective

Enable Git integration.

Open Fabric Dev Workspace

Go to:

Fabric-Dev-Workspace

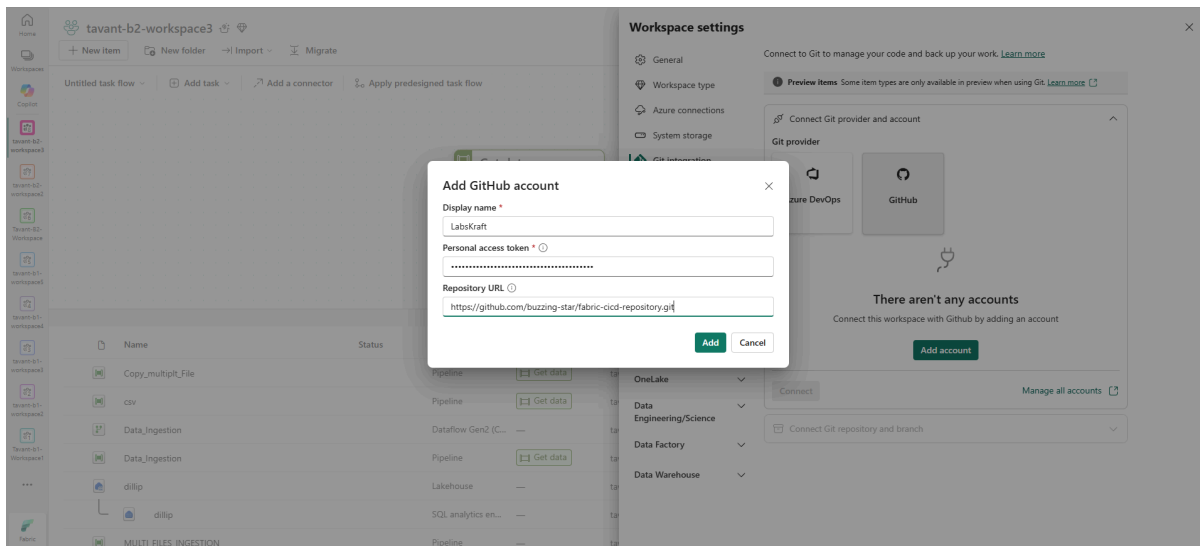
Open Workspace Settings

Click:

Workspace Settings → Git Integration

Configure Git Integration

Setting	Value
Git Provider	Git Repository
Organization	Your Organization
Repository	Fabric-CICD-Project
Branch	main
Folder	/fabric-artifacts



Authenticate

Sign in with Git Repository account.

Click Connect and Sync

Expected Output

Workspace connected to Git successfully

Step 6 — Create Sample Fabric Artifacts

Objective

Create artifacts that will be version controlled.

Create Lakehouse

Navigate

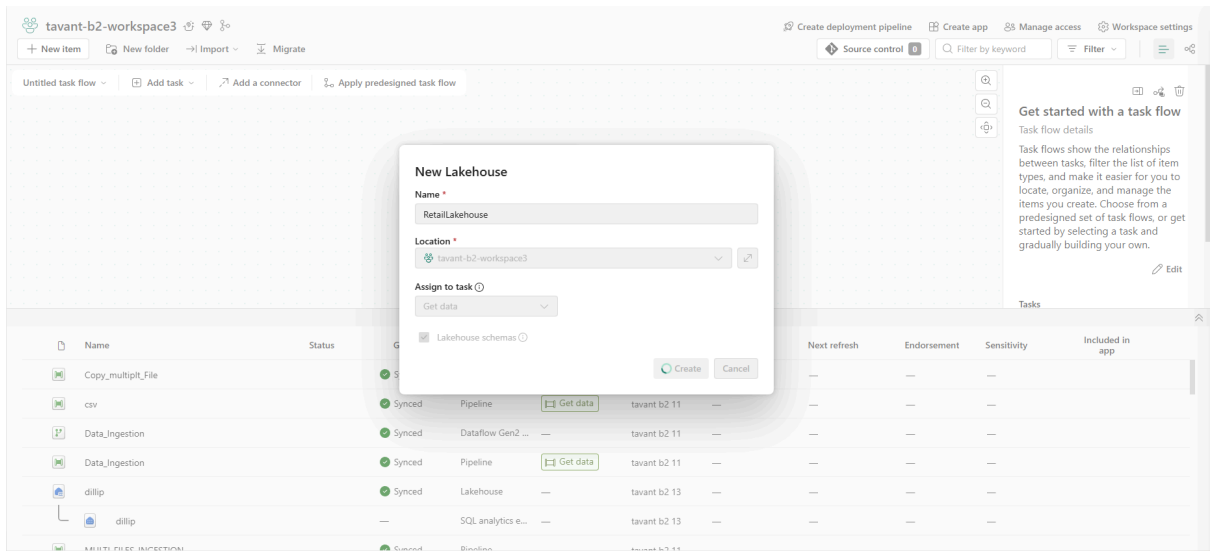
New → Lakehouse

Configure

Setting	Value
Name	RetailLakehouse

Expected Output

Lakehouse created successfully



Create Notebook

Navigate

New → Notebook

Notebook Name

sales_analysis_notebook

Add Sample PySpark Code

```
from pyspark.sql import SparkSession
```

```
sales_data = [  
    (1, "Laptop", 50000),  
    (2, "Phone", 30000),  
    (3, "Tablet", 20000)  
]
```

```
columns = ["id", "product", "sales"]
```

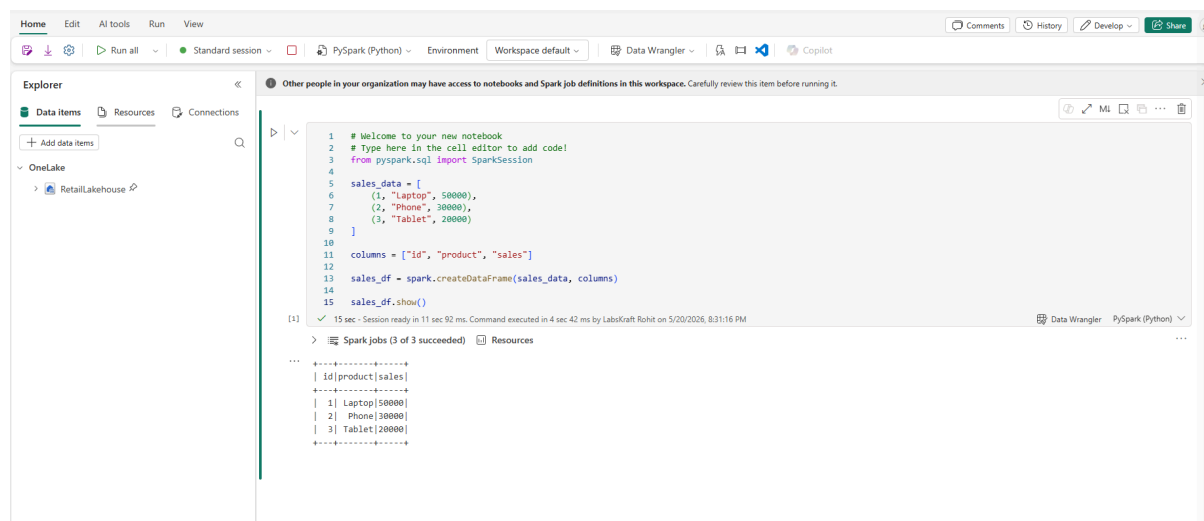
```
sales_df = spark.createDataFrame(sales_data, columns)
```

```
sales_df.show()
```

Run Notebook

Expected Output

```
+---+-----+-----+
|id|product|sales|
+---+-----+-----+
|1 |Laptop |50000|
|2 |Phone  |30000|
|3 |Tablet  |20000|
+---+-----+-----+
```



Create Data Pipeline

Navigate

New → Data Pipeline

Configure

Setting	Value
Name	RetailPipeline

Add Activities

Add:

- Notebook Activity
 - Copy Data Activity
-

Save Pipeline

Expected Output

Pipeline saved successfully

The screenshot displays the Microsoft Fabric interface, specifically the 'RetailPipeline' artifact. The top navigation bar includes 'Home', 'Activities', 'Run', and 'View'. The main workspace shows a pipeline diagram with a 'Notebook' activity (containing 'Notebook1') and a 'Copy data' activity (containing 'Copy data1'). The 'Parameters' tab is active, showing a '+ New' button. On the right, a 'Pipeline validation output' panel indicates 'Your pipeline has been validated. No errors were found.' Below this, the 'Pipeline run details' panel shows the run status as 'Succeeded' with a green checkmark. The 'Activity runs' section at the bottom lists the execution of 'Notebook1' with a status of 'Succeeded' and a duration of 44s.

Pipeline validation output

Your pipeline has been validated.
No errors were found.

Pipeline run details

Owner: LabsKraft Rohit
Submitted by: LabsKraft Rohit
Start time: 5/20/2026, 10:40:51 PM
End time: 5/20/2026, 10:41:46 PM
Status: **Succeeded**
Parameters: 0
Pipeline run ID: ca5452a2-2345-4b40-bb11-1938f548e372

Activity runs

Activity name	Activity status	Run start	Duration	Input	Output
Notebook1	Succeeded	5/20/2026, 10:41:01 PM	44s		

Step 7 — Commit Changes to Git

Objective

Push Fabric artifacts into Git repository.

Open Source Control Panel

Inside workspace:

Source Control

Review Pending Changes

You will see:

- Lakehouse
- Notebook
- Pipeline

The screenshot displays the Databricks Source Control interface for a workspace named 'tavant-b2-workspace3'. The main area shows a 'Get data' task flow with 17 items. A sidebar on the right, titled 'Source control', shows the current branch as 'Fabric-Pipeline' and lists pending changes. The changes table includes items like 'RetailLakehouse', 'RetailPipeline', and 'Notebook 4', all with a status of 'Synced'. At the bottom, there are 'Commit' and 'Undo' buttons.

Name	Status	Git status	Type	Task	Owner	Refrest
Copy_multitpit_File	Synced	Synced	Pipeline	Get data	tavant b2 12	—
csv	Synced	Synced	Pipeline	Get data	tavant b2 11	—
Data_Ingestion	Synced	Synced	Dataflow Gen2 (CL...	—	tavant b2 11	—
Data_Ingestion	Synced	Synced	Pipeline	Get data	tavant b2 11	—
dillip	Synced	Synced	Lakehouse	—	tavant b2 13	—
dillip	—	—	SQL analytics end...	—	tavant b2 13	—

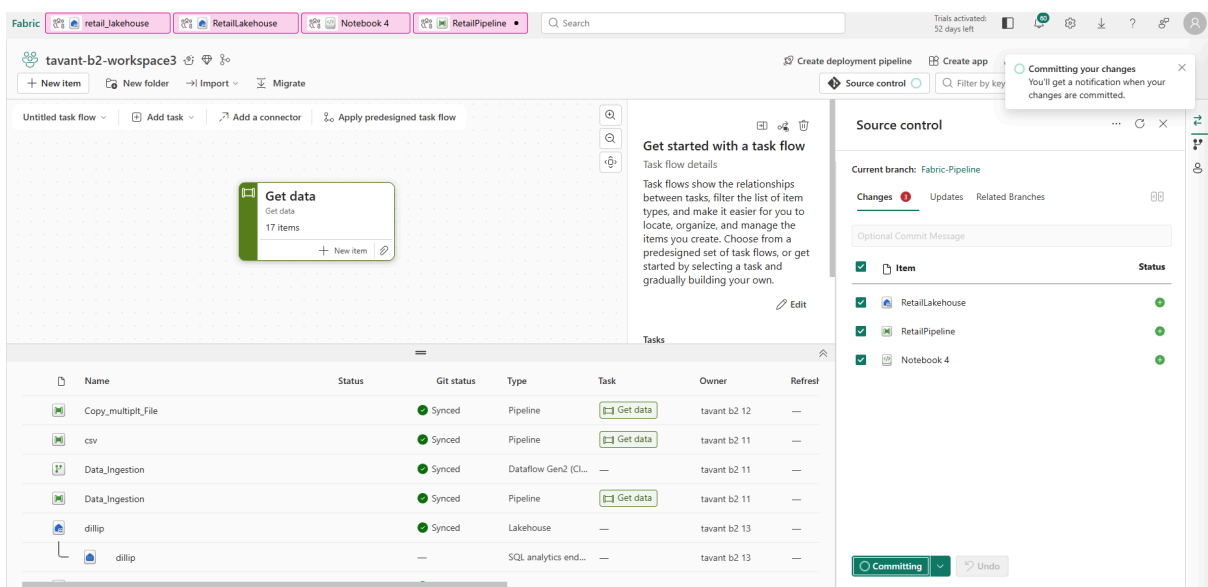
Add Commit Message

Initial Fabric artifacts commit

Commit Changes

Click:

Commit to Git



Step 8 — Validate Git Repository

Objective

Verify artifacts are stored in repository.

Open Git Repository Repository

Navigate:

Repository → Files

Verify Artifact Files

You should see:

/fabric-artifacts

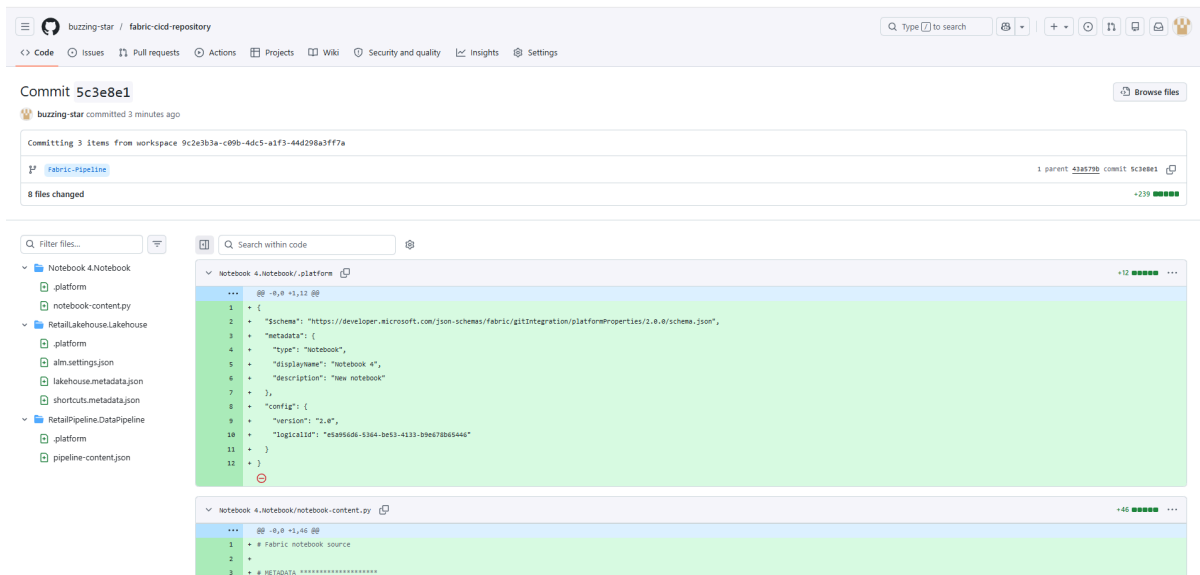
 /Lakehouse

 /Notebook

 /Pipeline

Expected Output

Fabric artifacts visible in Git repository



Step 9 — Modify Existing Artifact

Objective

Understand version control workflow.

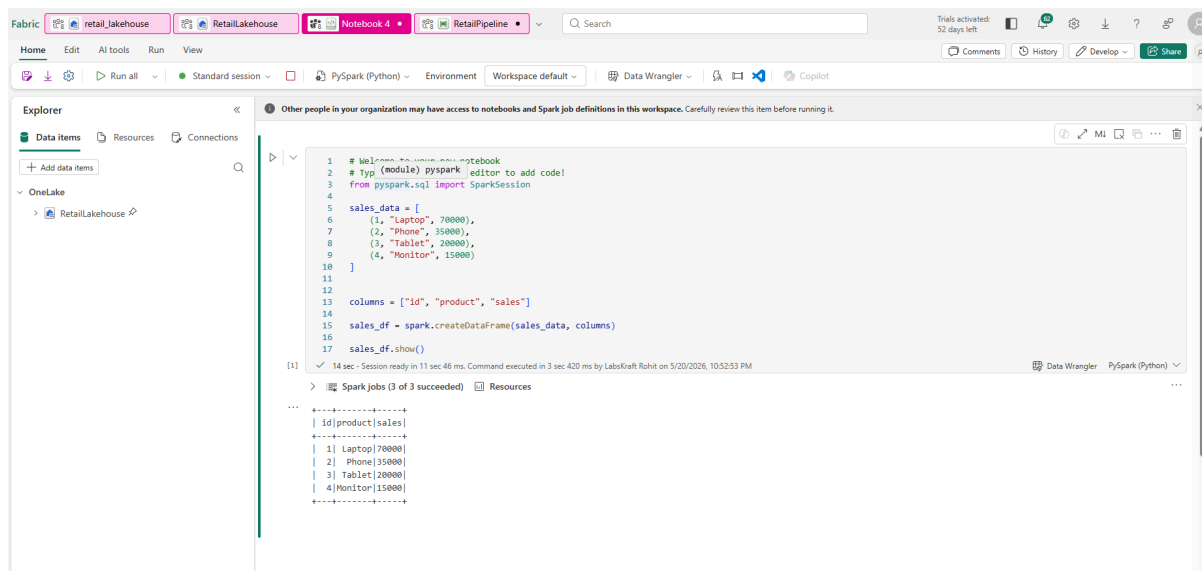
Open Notebook

Modify code:

```
sales_data = [
    (1, "Laptop", 50000),
    (2, "Phone", 30000),
    (3, "Tablet", 20000),
    (4, "Monitor", 15000)
```

]

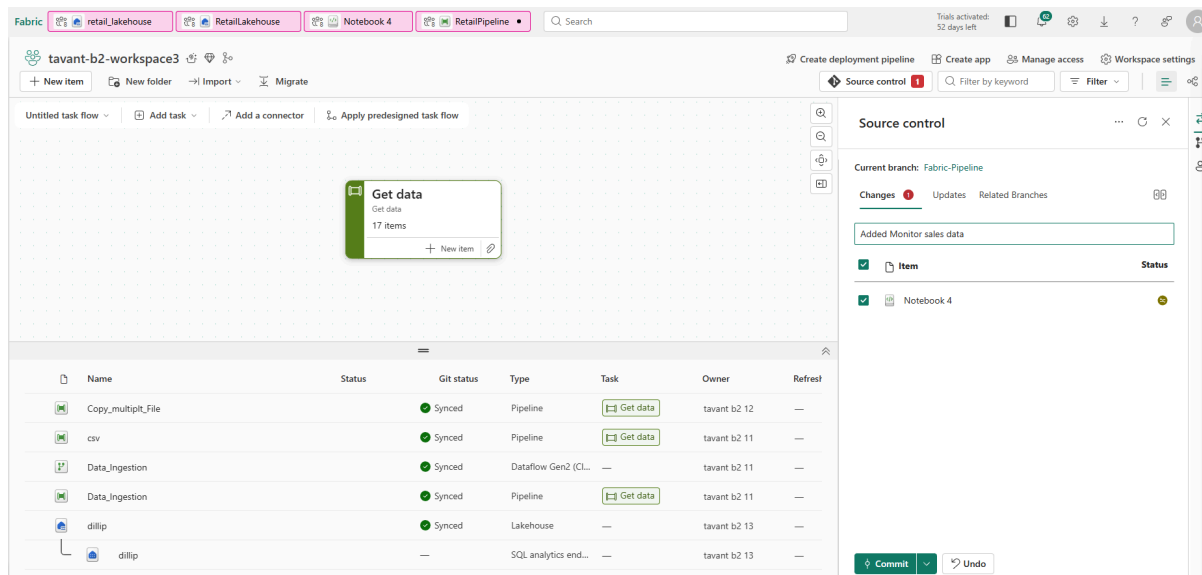
Save Notebook



Commit Updated Changes

Commit message:

Added Monitor sales data



Expected Output

New version committed successfully

Step 10 — View Git History

Objective

Track artifact versions.

Open Repository History

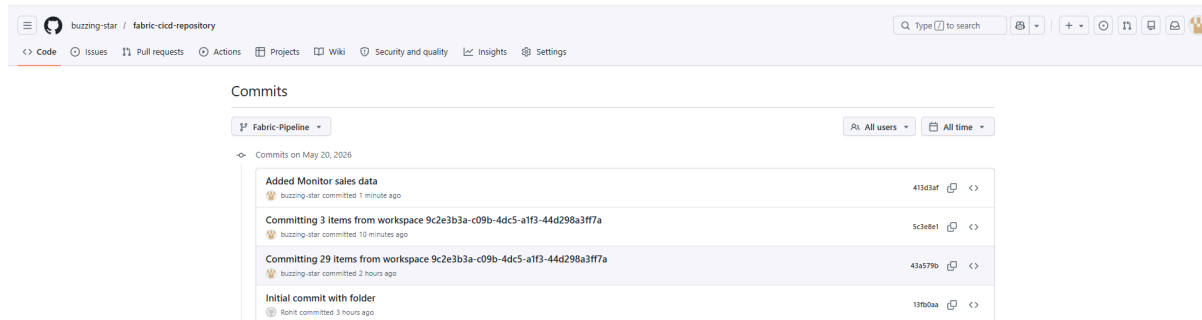
Navigate:

Repository → Commit History

Verify Commit History

You should see:

Initial Fabric artifacts commit
Added Monitor sales data



Expected Output

Git history visible successfully

Step 11 — Create Fabric Deployment Pipeline

Objective

Create CI/CD deployment workflow.

Open Deployment Pipelines

Navigate:

Workspaces → Deployment Pipelines

Create Pipeline

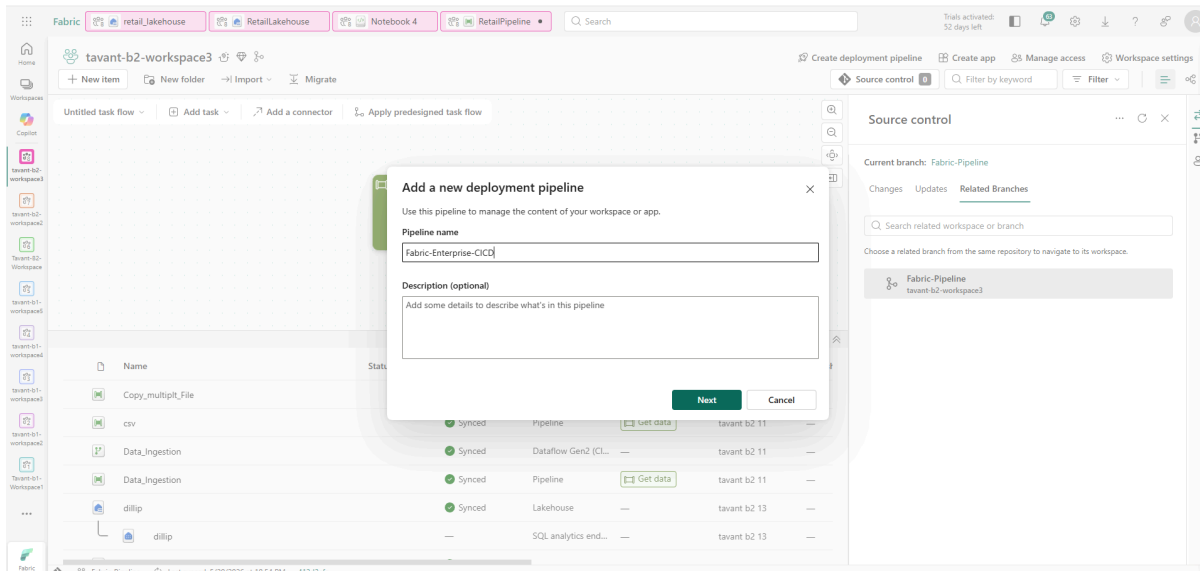
Click:

Create Pipeline

Configure Pipeline

Setting	Value
Pipeline Name	Fabric-Enterprise-CICD

Click Create



Expected Output

Deployment pipeline created successfully

Step 12 — Assign Workspaces to Pipeline

Objective

Map Dev and Prod environments.

Configure Development Stage

Assign:

Fabric-Dev-Workspace

Configure Production Stage

Assign:

Fabric-Prod-Workspace

Expected Output

Workspaces assigned successfully

Fabric-Enterprise-CICD

New Deployment pipelinesDeployment historyManage AccessSettings

Development
tavant-b2-workspace3

Production
tavant-b2-workspace2

Development

Deploy fromProduction

Deploy

Flat-ListSelect relatedFilter by keywordFilter

Selected stage item	Type	Compared to source	Source stage item
Copy_multipl_File	Pipeline	—	—
csv	Pipeline	—	—
Data_ingestion	Dataflow Gen2 (CI/CD) (Preview)	—	—
Data_ingestion	Pipeline	—	—

Step 13 — Deploy Dev to Production

Objective

Promote artifacts from Dev to Prod.

Open Deployment Pipeline

Select:

Fabric-Enterprise-CICD

Start Deployment

Click:

Deploy to Production

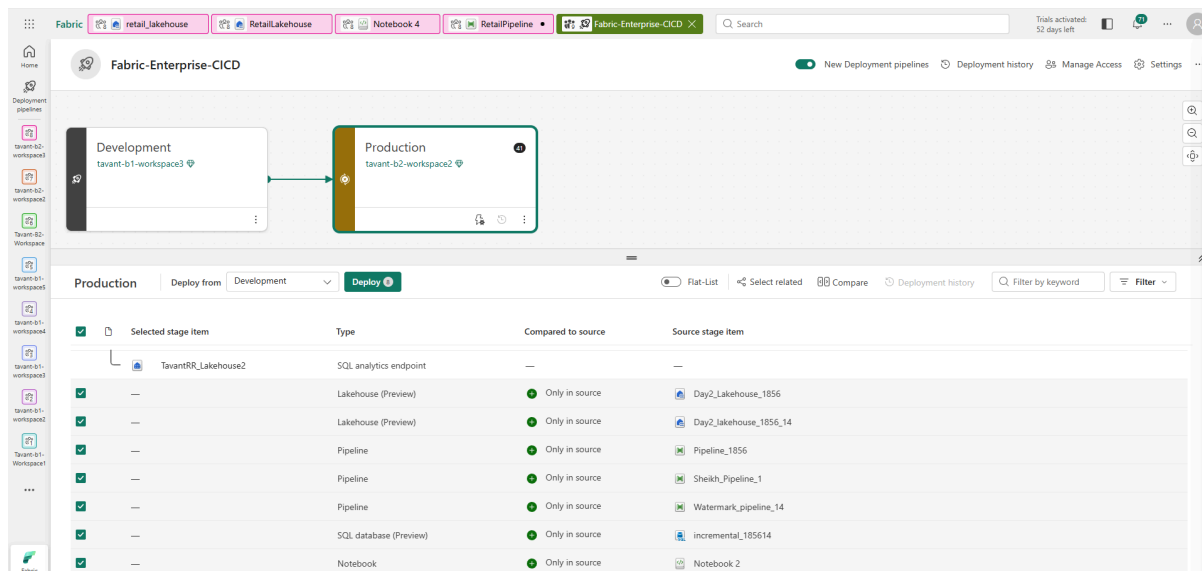
The screenshot displays the Fabric Enterprise CICD deployment pipeline interface. The top navigation bar includes tabs for various workspaces: Fabric, retail_lakehouse, RetailLakehouse, Notebook 4, RetailPipeline, and Fabric-Enterprise-CICD (selected). The main area shows a deployment pipeline with two stages: 'Development' (tavant-b2-workspace3) and 'Copying the content to this workspace...' (tavant-b2-workspace2). The 'Production' tab is selected, showing a table of deployment items.

Selected stage item	Type	Compared to source	Source stage item
Day2_Lakehouse_1856	Lakehouse (Preview)	—	—
Day2_Lakehouse_1856	SQL analytics endpoint	—	—
Day2_Lakehouse_1856_14	Lakehouse (Preview)	—	—
Day2_Lakehouse_1856_14	SQL analytics endpoint	—	—

Review Deployment Changes

You will see:

- Notebook
- Lakehouse
- Pipeline

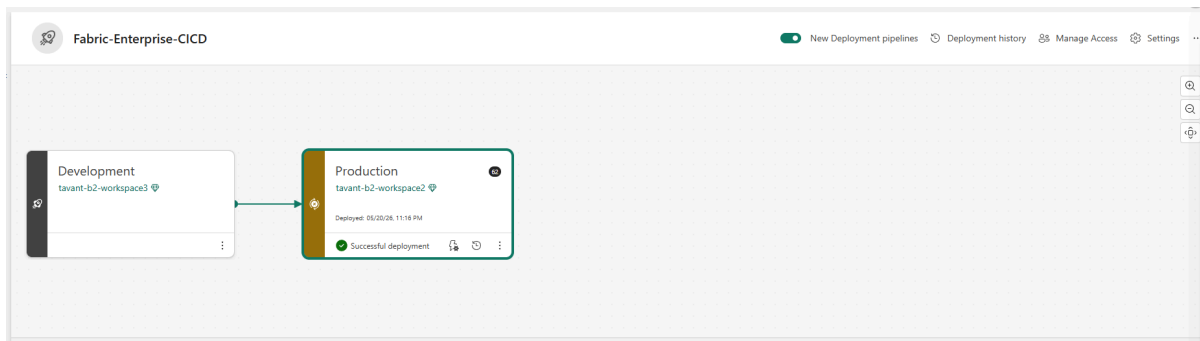


Click Deploy

Deployment starts.

Expected Output

Deployment completed successfully



Step 14 — Validate Production Workspace

Objective

Verify deployment success.

Open Production Workspace

Navigate:

Fabric-Prod-Workspace

Verify Artifacts

You should see:

-
- RetailLakehouse
 - sales_analysis_notebook
 - RetailPipeline
-

Run Notebook in Production

Execute notebook.

Expected Output

Notebook executed successfully in Production

Step 15 — Simulate Enterprise Change Management

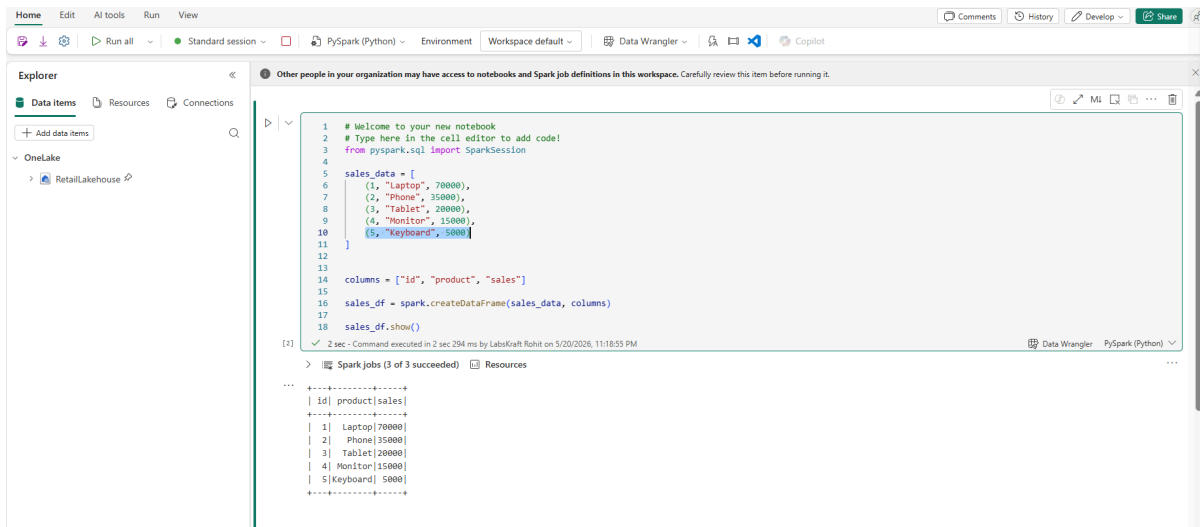
Objective

Understand real-world DevOps lifecycle.

Make Another Change in Dev

Modify notebook:

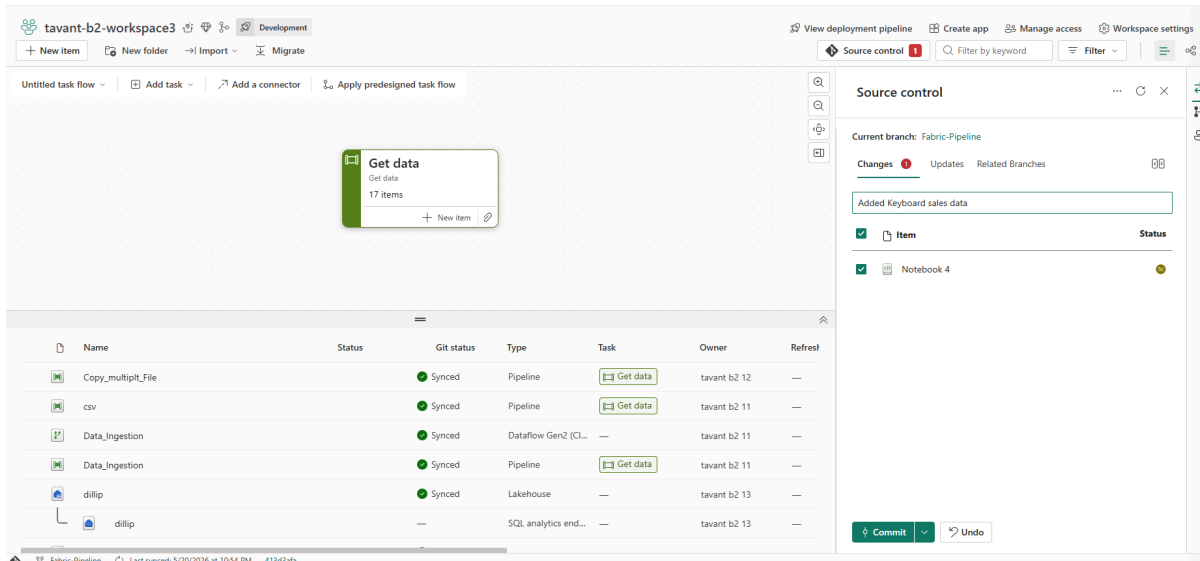
(5, "Keyboard", 5000)



Commit Changes

Commit message:

Added Keyboard sales data

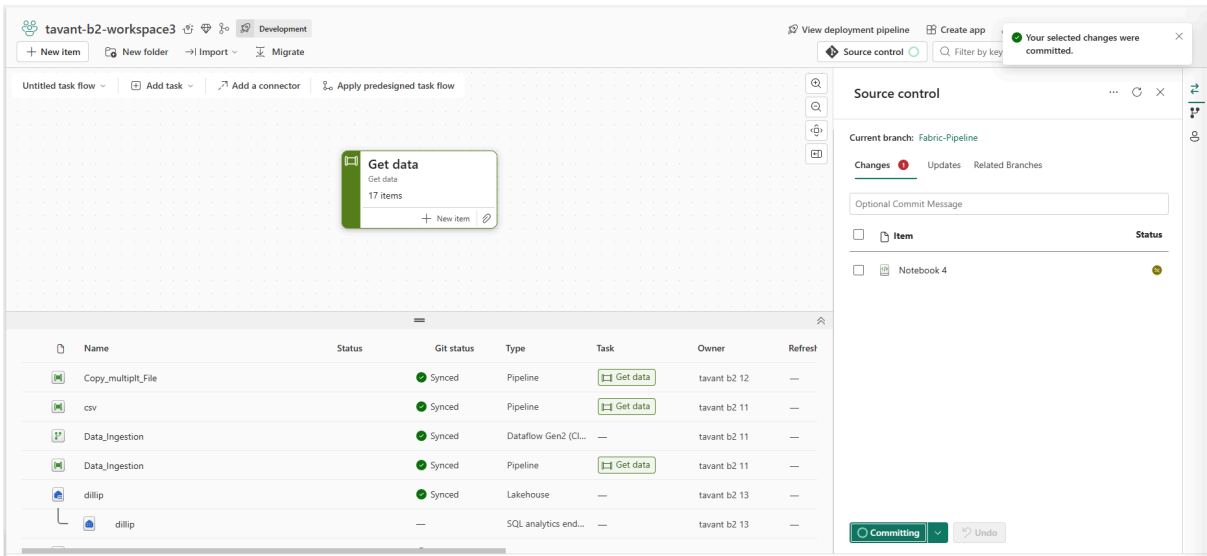


Go back to workspace : Open: Source Control
You will see: Modified Notebook pending changes.

Add Commit Message

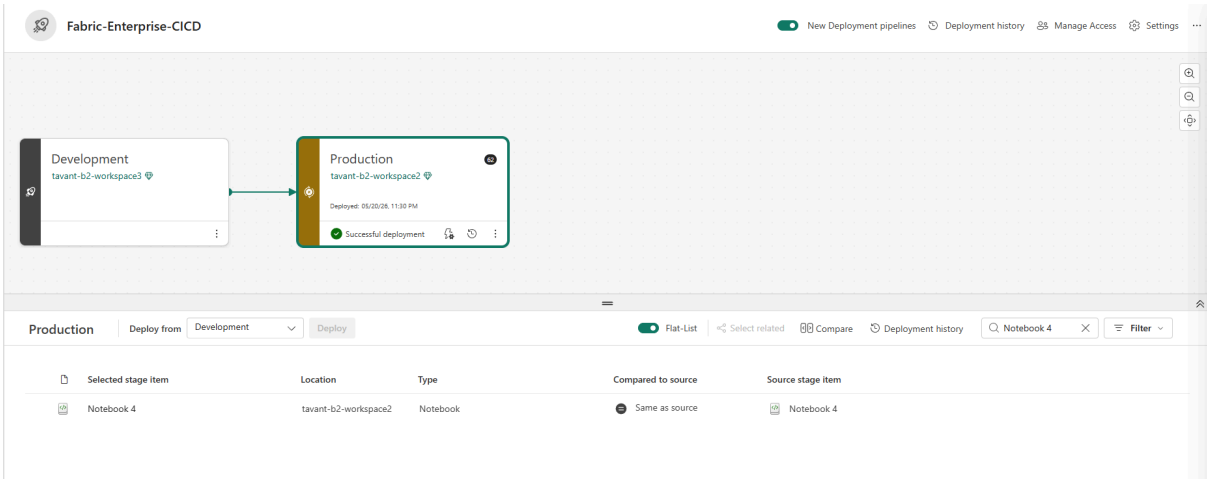
Example Added Keyboard sales data

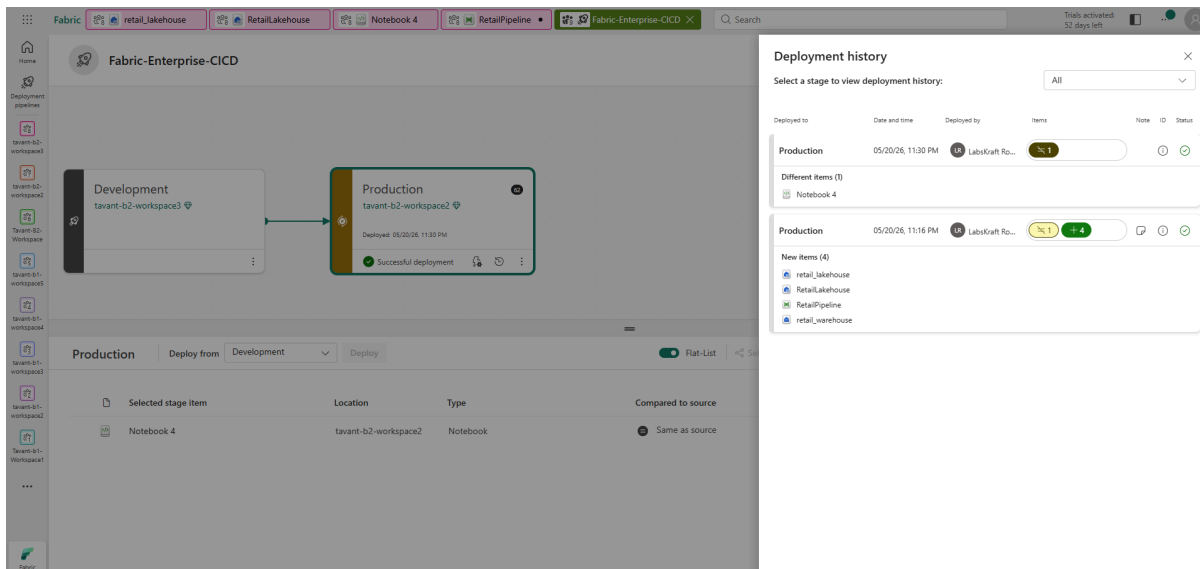
Click: Commit



Deploy Again

Deploy updated version from Dev to Prod.





Validate Production

Verify latest data is visible.

Expected Output

Updated artifacts deployed successfully

Step 16 — Rollback Scenario Understanding

Objective

Understand enterprise rollback strategy.

Rollback Concept

If deployment introduces issue:

- Open Git history
 - Revert previous commit
 - Sync workspace
 - Redeploy stable version
-

Enterprise Best Practices

Recommended Branch Strategy

Branch	Purpose
main	Production stable
dev	Development
feature/*	Feature development
hotfix/*	Production fixes

Recommended Environment Strategy

Environment	Purpose
Dev	Development
Test	QA testing
UAT	User acceptance
Prod	Production
